

Tribhuvan University
Faculty of Humanities and Social Science
Prithvi Narayan Campus, Pokhara

Lab Report on DBMS (CACS255)



BCA 4th Sem
[2079 Batch]

Submitted By

Name: Deep Darshan Thapa

Roll No: 04

Batch: 2079

Submitted To

Devilal Timilsina

BCA Program, PNC

Index

S.N.	Objective	Date of submission	Remarks
1.			
2.			
3.			
4.			
5.			
6.			
7.			
8.			
9.			
10.			
11.			

LAB 1

Objective: To Design A Simple Database

THEORY

A database is a collection of structured data that can be accessed or stored in a computer system. It is usually managed through a Database Management System (DBMS), a software used to manage data. There are different types of databases, and relational databases, such as MySQL, are particularly popular for their ability to manage data through tables and relationships.

Demonstration

Purpose of creating database is defined first. "CREATE" command is used to create database.

```
Setting environment for using XAMPP for Windows.
user@DESKTOP-F7BSI9A c:\xampp
# mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 8
Server version: 10.4.24-MariaDB mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> create database dmslab;
Query OK, 1 row affected (0.004 sec)

MariaDB [(none)]> use dmslab;
Database changed
MariaDB [dmslab]>
```

Based on purpose the main entities and their attributes are identified

```
MariaDB [(none)]> use dmslab;
Database changed
MariaDB [dmslab]> create table students (stud_id int primary key,
-> name varchar(50) not null,
-> address varchar (70) not null,
-> contact int (10) not null);
Query OK, 0 rows affected (0.028 sec)

MariaDB [dmslab]>
```

Data was added to each table using INSERT INTO statements

```
MariaDB [dmslab]> insert into students values ('1', 'Deep', 'pokhara', '9876543210');
Query OK, 1 row affected, 1 warning (0.019 sec)

MariaDB [dmslab]>
```

To describe the structure of table “Desc” statement was used

```
MariaDB [dmslab]> desc students;
```

Field	Type	Null	Key	Default	Extra
stud_id	int(11)	NO	PRI	NULL	
name	varchar(50)	NO		NULL	
address	varchar(70)	NO		NULL	
contact	int(10)	NO		NULL	

```
4 rows in set (0.017 sec)
```

To display content from table “select” statement was used

```
MariaDB [dmslab]> select * from students;
```

stud_id	name	address	contact
1	Deep	pokhara	2147483647

```
1 row in set (0.002 sec)
```

To delete database “DROP DATABASE database_name;” command is used

```
MariaDB [dmslab]> drop database dmslab;
Query OK, 1 row affected (0.015 sec)

MariaDB [(none)]>
```

Conclusion

In this lab the objective was to design a simple database. We successfully designed a simple database using basic operations.

LAB 2

Objective: To query and manipulate data in database

THEORY

Manipulating and querying data are essential aspects of working with relational databases. Data Manipulation Language (DML) in SQL includes commands to insert, update, delete, and retrieve data from tables, enabling users to manage and analyze data effectively.

Demonstration

Basic query

```
MariaDB [dmslab]> SELECT * FROM students;
+-----+-----+-----+-----+
| id | name | address | contact |
+-----+-----+-----+-----+
| 1 | Deep | Pokhara | 2147483647 |
+-----+-----+-----+-----+
1 row in set (0.003 sec)
```

Conditional query

```
MariaDB [dmslab]> SELECT name, contact FROM students WHERE address = 'Pokhara';
+-----+-----+
| name | contact |
+-----+-----+
| Deep | 2147483647 |
+-----+-----+
1 row in set (0.003 sec)
```

To modify existing records

```
MariaDB [dmslab]> UPDATE students
-> SET contact = 97650000000 WHERE id=1;
Query OK, 0 rows affected, 1 warning (0.006 sec)
Rows matched: 1 Changed: 0 Warnings: 1
```

To delete table

```
MariaDB [dmslab]> DELETE FROM students
-> WHERE id = 1
-> ;
Query OK, 1 row affected (0.004 sec)
```

Conclusion

In this lab the objective was to query and manipulate data in database. Querying and manipulating data are fundamental operations in database management, enabling efficient handling of large datasets. Mastery of these techniques is essential for effectively managing relational databases and driving data-driven decisions.

LAB 3

Objective: To Retrieve Data From Different Tables Using Joins

Theory

Data retrieval in MySQL is primarily done using the SELECT statement. Data retrieval from different tables using joins in MySQL allows you to combine rows from two or more tables based on a related column between them.

Demonstration

To demonstrate joins tables are created

```
MariaDB [dmslab]> INSERT INTO Employees (EmployeeID, Name, DepartmentID) VALUES
-> (1, 'Deep', 101),
-> (2, 'Deepa', 102),
-> (3, 'Deepak', 103),
-> (4, 'Manoj', NULL),
-> (5, 'Ram', 104);
Query OK, 5 rows affected (0.008 sec)
Records: 5 Duplicates: 0 Warnings: 0

MariaDB [dmslab]>
```

```
MariaDB [dmslab]> CREATE TABLE Departments (
-> DepartmentID INT PRIMARY KEY,
-> DepartmentName VARCHAR(50) );
Query OK, 0 rows affected (0.017 sec)

MariaDB [dmslab]> INSERT INTO Departments (DepartmentID, DepartmentName) VALUES
-> (101, 'Human Resources'),
-> (102, 'Finance'),
-> (103, 'Engineering'),
-> (105, 'Sales');
Query OK, 4 rows affected (0.007 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

Here are the main types of joins:

1. Inner Join

```
MariaDB [dmslab]>
MariaDB [dmslab]> SELECT Employees.Name AS EmployeeName, Departments.DepartmentName
-> FROM Employees
-> INNER JOIN Departments ON Employees.DepartmentID = Departments.DepartmentID;
+-----+-----+
| EmployeeName | DepartmentName |
+-----+-----+
| Deep         | Human Resources |
| Deepa        | Finance         |
| Deepak       | Engineering     |
+-----+-----+
3 rows in set (0.007 sec)
```

2. Left Join

```
MariaDB [dmslab]> Select Employees.Name AS EmployeeName, Departments.DepartmentName
-> from Employees
-> left join Departments on Employees.DepartmentID = Departments.DepartmentID;
```

EmployeeName	DepartmentName
Deep	Human Resources
Deepa	Finance
Deepak	Engineering
Manoj	NULL
Ram	NULL

```
5 rows in set (0.003 sec)
```

3. Right Join

```
MariaDB [dmslab]> select Employees.Name AS EmployeeName, Departments.DepartmentName
-> from Employees
-> RIGHT JOIN Departments ON Employees.DepartmentID = Departments.DepartmentID;
```

EmployeeName	DepartmentName
Deep	Human Resources
Deepa	Finance
Deepak	Engineering
NULL	Sales

```
4 rows in set (0.000 sec)
```

```
MariaDB [dmslab]>
```

4. Cross Join

```
MariaDB [dmslab]> SELECT Employees.Name AS EmployeeName, Departments.DepartmentName
-> FROM Employees
-> CROSS JOIN Departments;
```

EmployeeName	DepartmentName
Deep	Human Resources
Deep	Finance
Deep	Engineering
Deep	Sales
Deepa	Human Resources
Deepa	Finance
Deepa	Engineering
Deepa	Sales
Deepak	Human Resources
Deepak	Finance
Deepak	Engineering
Deepak	Sales
Manoj	Human Resources
Manoj	Finance
Manoj	Engineering
Manoj	Sales
Ram	Human Resources
Ram	Finance
Ram	Engineering
Ram	Sales

```
20 rows in set (0.000 sec)
```


Conclusion

Using SQL joins, we can effectively retrieve and combine related data from multiple tables. Each type of join serves specific use cases, enhancing the flexibility and power of relational databases for complex queries.

LAB 4

Objective: To study different data types in mysql

Theory

In MySQL, data types define the kind of data a column can store in a table. Choosing the correct data type for each column is critical in designing efficient databases. The MySQL database management system offers various categories of data types, including numeric, string, date/time, and binary types.

Demonstration

1. First table name `datatypes` is created that contains different types

```
MariaDB [dbms_lab]> CREATE TABLE DATATYPES (  
->  
-> -- Numeric Data Types  
-> int_column INT,  
-> tinyint_column TINYINT,  
-> smallint_column SMALLINT,  
-> mediumint_column MEDIUMINT,  
-> bigint_column BIGINT,  
-> float_column FLOAT,  
-> double_column DOUBLE,  
-> decimal_column DECIMAL(10, 2),  
-> unsigned_int_column INT UNSIGNED,  
->  
-> -- String Data Types  
-> char_column CHAR(10),  
-> varchar_column VARCHAR(50),  
-> text_column TEXT,  
-> blob_column BLOB,  
-> enum_column ENUM('option1', 'option2', 'option3'),  
-> set_column SET('red', 'green', 'blue'),  
->  
-> -- Date and Time Data Types  
-> date_column DATE,  
-> datetime_column DATETIME,  
-> timestamp_column TIMESTAMP,  
-> time_column TIME,  
-> year_column YEAR,  
->  
-> -- Other Data Types  
-> bool_column BOOLEAN  
-> );  
Query OK, 0 rows affected (0.009 sec)
```

2. Using `desc <table_name>` command we can view datatypes used in the table

```
MariaDB [dbms_lab]> desc DATATYPES;
```

Field	Type	Null	Key	Default	Extra
int_column	int(11)	YES		NULL	
tinyint_column	tinyint(4)	YES		NULL	
smallint_column	smallint(6)	YES		NULL	
mediumint_column	mediumint(9)	YES		NULL	
bigint_column	bigint(20)	YES		NULL	
float_column	float	YES		NULL	
double_column	double	YES		NULL	
decimal_column	decimal(10,2)	YES		NULL	
unsigned_int_column	int(10) unsigned	YES		NULL	
char_column	char(10)	YES		NULL	
varchar_column	varchar(50)	YES		NULL	
text_column	text	YES		NULL	
blob_column	blob	YES		NULL	
enum_column	enum('option1','option2','option3')	YES		NULL	
set_column	set('red','green','blue')	YES		NULL	
date_column	date	YES		NULL	
datetime_column	datetime	YES		NULL	
timestamp_column	timestamp	NO		current_timestamp()	on update current_timestamp()
time_column	time	YES		NULL	
year_column	year(4)	YES		NULL	
bool_column	tinyint(1)	YES		NULL	

21 rows in set (0.014 sec)

Conclusion

In this lab the objective was to study different datatypes in mysql. We learned it by creating table containing different datatypes.

Lab 5

Objective: To Implement Referential Integrity using Primary Key and Foreign Key Constraints

Theory

Referential integrity is a concept used in relational databases to ensure that relationships between tables remain consistent. Specifically, it ensures that foreign keys in a child table correctly reference primary keys in a parent table. In MySQL, Primary Key and Foreign Key constraints are used to enforce referential integrity.

Primary Key: A unique identifier for each record in a table.

Foreign Key: A column (or set of columns) in one table that references the primary key of another table to establish a relationship.

Demonstration

First student table is created.

```
MariaDB [dmslab]> CREATE TABLE Students (  
  ->   StudentID INT NOT NULL PRIMARY KEY,  
  ->   Name VARCHAR(50),  
  ->   Age INT  
  -> );  
Query OK, 0 rows affected (0.014 sec)
```

Next, grade table is created.

```
MariaDB [dmslab]> CREATE TABLE Grades (   GradeID INT NOT NULL PRIMARY KEY,   StudentID INT,   Subject VARCHAR(50),  
  Marks INT,   FOREIGN KEY (StudentID) REFERENCES Students(StudentID));  
Query OK, 0 rows affected (0.017 sec)
```

Insert values in the table

```
MariaDB [dmslab]> INSERT INTO Students (StudentID, Name, Age)  
  -> VALUES (1, 'Deep', 16),  
  ->         (2, 'Drsn', 61);  
Query OK, 2 rows affected (0.019 sec)
```

```
MariaDB [dmslab]> INSERT INTO Grades (GradeID, StudentID, Subject, Marks)  
  -> VALUES (1, 1, 'Math', 85),  
  ->         (2, 2, 'Dms', 90);  
Query OK, 2 rows affected (0.006 sec)
```

Query to fetch data from both table

```
MariaDB [dmslab]> SELECT Students.Name, Grades.Subject, Grades.Marks  
-> FROM Students  
-> INNER JOIN Grades ON Students.StudentID = Grades.StudentID;
```

Name	Subject	Marks
Deep	Math	85
Drsn	Dms	90

Conclusion

This lab demonstrates how to enforce referential integrity in MySQL using primary and foreign key constraints. It ensures that the Grades table references valid StudentID values from the Students table, maintaining the consistency of related data.

Lab 6

Objective: To Develop Stored Procedures And Alter Store Procedures.

Theory

A **stored procedure** is a precompiled set of SQL statements stored in the database that can be executed as a single unit. Stored procedures improve performance, reusability, and security in database management.

Key operations:

1. **Create Stored Procedure:** Define a reusable SQL block.
2. **Call Stored Procedure:** Execute the procedure.
3. **Alter Stored Procedure:** Modify an existing procedure.

Demonstration

stored procedure named GetStudentDetails is created hat retrieves all records from the Students table.

In MySQL, the *DELIMITER* command changes the default statement delimiter (semicolon ;) to another specified string, such as \$\$\$. This is especially useful when writing stored procedures, functions, or triggers that include multiple SQL statements separated by semicolons.

```
MariaDB [dmslab]> DELIMITER $$
MariaDB [dmslab]>
MariaDB [dmslab]> CREATE PROCEDURE GetStudentDetails()
  -> BEGIN
  ->     SELECT * FROM Students;
  -> END $$$
Query OK, 0 rows affected (0.018 sec)
```

```
MariaDB [dmslab]>
MariaDB [dmslab]> DELIMITER ;
```

```
MariaDB [dmslab]> CALL GetStudentDetails();
+-----+-----+-----+
| StudentID | Name | Age |
+-----+-----+-----+
|          1 | Deep | 16 |
|          2 | Drsn | 61 |
+-----+-----+-----+
2 rows in set (0.006 sec)
```

```
Query OK, 0 rows affected (0.029 sec)
```

Conclusion

The implementation demonstrates how to create, call, and alter stored procedures in MySQL. Stored procedures enhance reusability and simplify complex query execution while providing flexibility to modify functionality as needed.

LAB 7

Objective: To Develop Stored Procedures With Various Parameters.

Theory

A stored procedure is a precompiled set of SQL statements stored in the database that can be executed as a single unit. Stored procedures improve performance, reusability, and security in database management. Parameters allow stored procedures to accept input values, making them more flexible and dynamic.

Key operations:

1. **Create Stored Procedure:** Define a reusable SQL block with parameters.
2. **Call Stored Procedure:** Execute the procedure with different input values.
3. **Alter Stored Procedure:** Modify an existing procedure to change parameters or functionality.

Demonstration

Create a Stored Procedure with IN Parameter

```
MariaDB [dmslab]> CREATE PROCEDURE GetStudentsByAge(IN min_age INT)
-> SELECT * FROM Students WHERE Age >= min_age;
Query OK, 0 rows affected (0.007 sec)
```

Create a Stored Procedure with OUT Parameter

```
MariaDB [dmslab]> CREATE PROCEDURE CountStudents(OUT total INT)
-> SELECT COUNT(*) INTO total FROM Students;
Query OK, 0 rows affected (0.008 sec)
```

Create a Stored Procedure with INOUT Parameter

```
MariaDB [dmslab]> CREATE PROCEDURE UpdateAndGetAge(INOUT student_id INT, OUT age INT)
-> BEGIN
->   UPDATE Students
->   SET Age = Age + 1
->   WHERE StudentID = student_id;
->   SELECT Age INTO age
->   FROM Students
->   WHERE StudentID = student_id;
-> END $$
Query OK, 0 rows affected (0.005 sec)
```

Call Procedure with IN Parameter


```
MariaDB [dmslab]> CALL GetStudentsByAge(21);
```

StudentID	Name	Age
2	Drsn	61

```
1 row in set (0.004 sec)
```

Call Procedure with OUT Parameter

```
MariaDB [dmslab]> SELECT @total_students AS TotalStudents;
```

TotalStudents
2

```
1 row in set (0.001 sec)
```

Call Procedure with INOUT Parameter

```
MariaDB [dmslab]> SELECT @student_id AS StudentID, @student_age AS UpdatedAge;
```

StudentID	UpdatedAge
1	NULL

```
1 row in set (0.000 sec)
```

Conclusion

The implementation demonstrates how to use **IN**, **OUT**, and **INOUT** parameters in MySQL stored procedures to pass and manipulate data efficiently. This approach enhances modularity and reusability in database management.

Lab 8

Objective: To Use Triggers In Stored Procedures

Theory

A trigger is a set of SQL statements that automatically execute or "fire" when specific database events occur, such as INSERT, UPDATE, or DELETE. Triggers can help maintain data integrity, enforce business rules, and automate tasks. Combining triggers with stored procedures allows for more complex operations and interactions within the database.

Key operations:

1. **Create Stored Procedure:** Define a reusable SQL block.
2. **Create Trigger:** Define a trigger that invokes the stored procedure.
3. **Test Trigger:** Execute the trigger by performing the specified event.
4. **Modify Trigger:** Change the trigger to demonstrate flexibility and adaptability

Demonstration

Create a Stored Procedure

```
MariaDB [dmslab]> CREATE PROCEDURE LogStudentAction(IN action_type VARCHAR(50), IN student_id INT)
-> BEGIN
->     INSERT INTO StudentLogs(ActionType, StudentID, ActionTime)
->     VALUES (action_type, student_id, NOW());
-> END $$
Query OK, 0 rows affected (0.005 sec)
```

Create a Trigger that Calls the Stored Procedure

```
MariaDB [dmslab]> CREATE TRIGGER AfterStudentInsert
-> AFTER INSERT ON Students
-> FOR EACH ROW
-> BEGIN
->     CALL LogStudentAction('INSERT', NEW.StudentID);
-> END $$
Query OK, 0 rows affected (0.025 sec)
```

Test the Trigger

```
MariaDB [dmslab]> INSERT INTO Students (Name, Age) VALUES ('Deep','18');
ERROR 1062 (23000): Duplicate entry '0' for key 'PRIMARY'
```

Modify the trigger

```
MariaDB [dmslab]> CREATE TRIGGER AfterStudentUpdate
-> AFTER UPDATE ON Students
-> FOR EACH ROW
-> BEGIN
->     CALL LogStudentAction('UPDATE', NEW.StudentID);
-> END $$
Query OK, 0 rows affected (0.033 sec)
```

Conclusion

This implementation demonstrates how to create and use triggers within stored procedures in MySQL. Triggers automate tasks and maintain data integrity by invoking stored procedures based on specific database events. This combination enhances database functionality and enforces business rules more efficiently.